

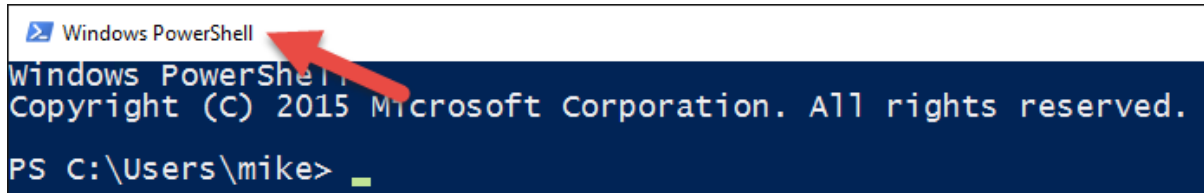


المدرسة الوطنية للحكاه الاصطناعى والرقمىة بركان
ⵜⴰⵍⴷⵓⴷⴰ ⵜⴰⵎⴰⵏⴷⴰⵢⵜ ⵜⴰⵎⴰⵏⴷⴰⵢⵜ ⵜⴰⵎⴰⵏⴷⴰⵢⵜ ⵜⴰⵎⴰⵏⴷⴰⵢⵜ
École Nationale de l'Intelligence Artificielle et du Digital Berkane

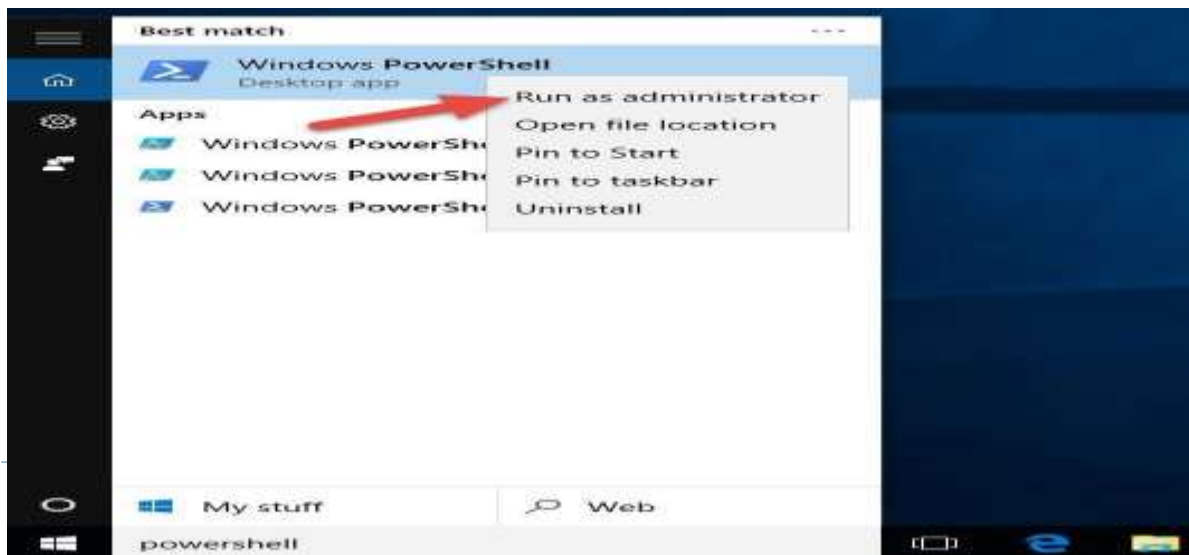
Power Shell en détail

Lancer PowerShell

- Pour lancer Powershell cliquer sur sur le raccourci « Windows PowerShell »

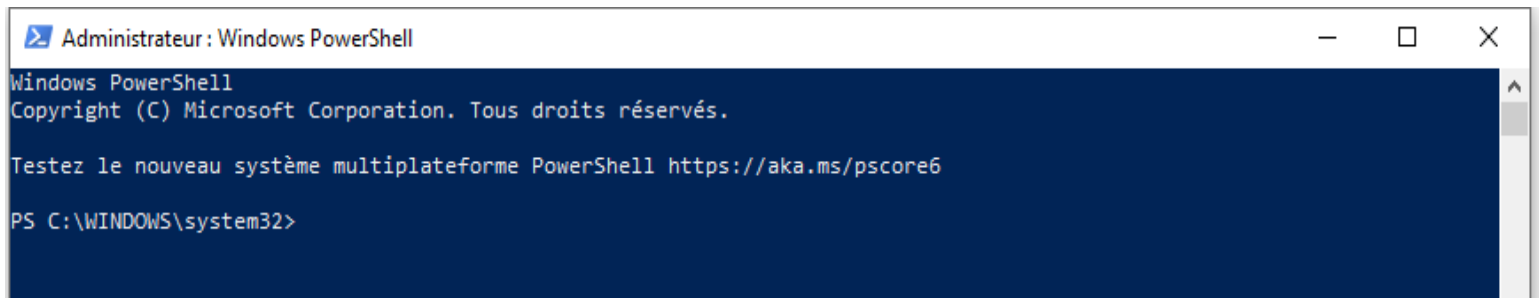


- Pour exécuter PowerShell en tant qu'utilisateur de domaine et **administrateur local** : Relancez la console PowerShell, mais cette fois-ci, cliquez avec le bouton droit sur le raccourci **Windows PowerShell**, puis sélectionnez **Exécuter en tant qu'administrateur**



Lancer PowerShell

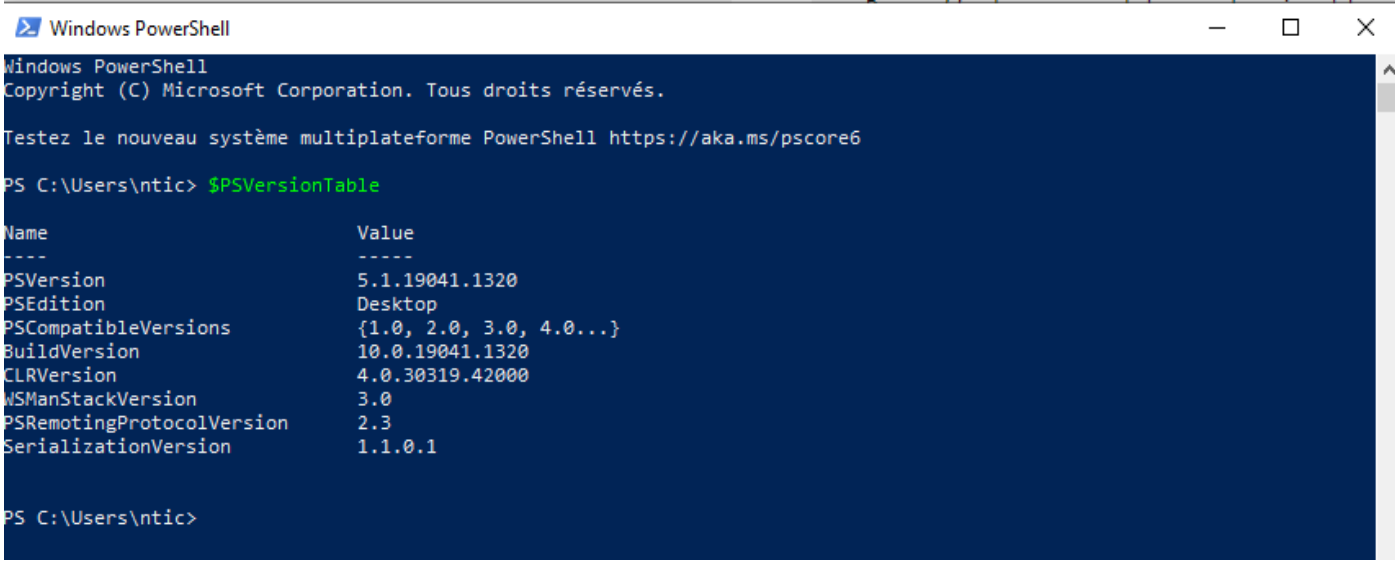
- Une fois que vous avez relancé PowerShell en tant qu'administrateur, la barre de titre doit indiquer « Administrateur : Windows PowerShell », comme illustré à la figure



Afficher PowerShell

- Dans PowerShell, il existe plusieurs variables automatiques qui stockent des informations d'état. L'une de ces variables est :

\$PSVersionTable



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Tous droits réservés.

Testez le nouveau système multiplateforme PowerShell https://aka.ms/pscore6

PS C:\Users\ntic> $PSVersionTable

Name                           Value
----                           -
PSVersion                      5.1.19041.1320
PSEdition                     Desktop
PSCompatibleVersions           {1.0, 2.0, 3.0, 4.0...}
BuildVersion                   10.0.19041.1320
CLRVersion                     4.0.30319.42000
WSManStackVersion              3.0
PSRemotingProtocolVersion      2.3
SerializationVersion           1.1.0.1

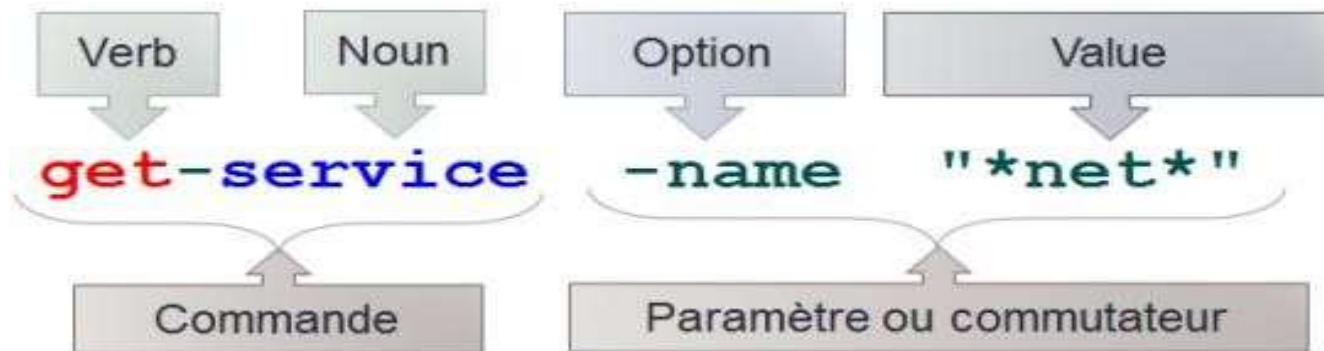
PS C:\Users\ntic>
```

- La sortie fournit des informations sur la version de PowerShell, ainsi que sur votre plateforme et votre édition

-
- **La structure des commandes**

Les applets de commandes “cmdlet”

- Dans le monde Powershell, les applets de commandes (cmdlet) sont composées d’une paire de la forme “**verbe**”-“**nom**” destiné à en faciliter la mémorisation.



Les applets de commandes “cmdlet”

- Le résultat renvoie une sorte de tableau avec des entêtes de colonnes, où chaque ligne représente un élément.



Trouver des commandes

- Vous pouvez voir la liste des verbes approuvés en utilisant l'applet de commande **Get-Verb**. Les verbes sont organisés par type d'activité et par fonction.

```
PS C:\Users\ntic> Get-Verb
```

Verb	Group
----	----
Add	Common
Clear	Common
Close	Common
Copy	Common
Enter	Common
Exit	Common
Find	Common
Format	Common
Get	Common
Hide	Common
Join	Common
Lock	Common
Move	Common
New	Common
Open	Common
Optimize	Common
Pop	Common
Push	Common
Redo	Common
Remove	Common
Rename	Common
Reset	Common
Resize	Common
Search	Common
Select	Common
Set	Common
Show	Common
Skip	Common
Split	Common
Step	Common
Switch	Common
Undo	Common
Unlock	Common
Watch	Common
Backup	Data
Checkpoint	Data
Compare	Data
Compress	Data
Convert	Data
ConvertFrom	Data
ConvertTo	Data

Trouver des commandes

Trois applets de commande principales vous permettent de découvrir les applets de commande existantes et ce qu'elles font :

- **Get-Command** : L'applet de commande Get-Command liste toutes les applets de commande disponibles sur votre système.
- **Get-Help** : utilisez la cmdlet de base Get-Help pour appeler un système d'aide intégré.
- **Get-Member** : Quand vous appelez une commande, la réponse est un objet qui contient de nombreuses propriétés.

Recherche des commandes en utilisant **Get-Command**

Avec Get-Command on utilise des indicateurs pour filtrer :

- **Noun** : L'indicateur -Noun cible la partie du nom de la commande. Autrement dit, il cible tout ce qui correspond à ce qui se trouve après le trait d'union (-). Voici un exemple :

```
Get-Command -Noun NETIPADD*
```

```
PS C:\Users\ntic> Get-Command -Noun NetIPADDR*

CommandType      Name                               Version      Source
-----
Function         Get-NetIPAddress                 1.0.0.0     NetTCPIP
Function         New-NetIPAddress                 1.0.0.0     NetTCPIP
Function         Remove-NetIPAddress              1.0.0.0     NetTCPIP
Function         Set-NetIPAddress                 1.0.0.0     NetTCPIP

PS C:\Users\ntic>
```

Cette commande recherche toutes les applets de commande dont la partie « substantif » commence par NETIPADD

Rechercher des commandes en utilisant **Get-Command**

- **Verb** : L'indicateur -Verb cible la partie du nom de la commande qui est associée au verbe. Vous pouvez combiner l'indicateur -Noun et l'indicateur -Verb, voici un exemple :

```
Get-Command -Verb Get -Noun NETIPADD*
```

```
PS C:\Users\ntic> Get-Command -Verb Get -Noun NETIPADD*

CommandType      Name                               Version      Source
-----
Function         Get-NetIPAddress                 1.0.0.0     NetTCPIP
PS C:\Users\ntic>
```

Get-Help

Une fois que l'on a repéré la commande qui nous intéresse grâce à **Get-Command**, on va en toute logique se poser la question suivante : **comment s'utilise cette commande PowerShell ?** C'est là qu'intervient le cmdlet "***Get-Help***" puisqu'il permet d'obtenir de l'aide.

```
PS C:\Users\ZITI> Get-Help Start-Service
```

NOM

```
Start-Service
```

RÉSUMÉ

```
Starts one or more stopped services.
```

SYNTAXE

```
Start-Service -DisplayName <System.String[]> [-Exclude <System.String[]>] [-Include <System.String[]>] [-PassThru] [-Confirm] [-WhatIf] [<CommonParameters>]
```

```
Start-Service [-InputObject] <System.ServiceProcess.ServiceController[]> [-Exclude <System.String[]>] [-Include <System.String[]>] [-PassThru] [-Confirm] [-WhatIf] [<CommonParameters>]
```

```
Start-Service [-Name] <System.String[]> [-Exclude <System.String[]>] [-Include <System.String[]>] [-PassThru] [-Confirm] [-WhatIf] [<CommonParameters>]
```

DESCRIPTION

The `Start-Service` cmdlet sends a start message to the Windows Service Controller for each of the specified services. If a service is already running, the message is ignored without error. You can specify the services by their service names or display names, or you can use the InputObject parameter to supply a service object that represents the services that you want to start.

Get-Member

Pour explorer une commande, il y a le cmdlet "*Get-Member*" et son alias "*gm*" qui sont à connaître. Cette commande va permettre d'obtenir tous les membres d'une commande, c'est-à-dire **toutes les propriétés et méthodes d'une commande**.

```
PS C:\Users\ZITI> Get-Service | Get-Member
```

```
TypeName : System.ServiceProcess.ServiceController
```

Name	MemberType	Definition
----	-----	-----
Name	AliasProperty	Name = ServiceName
RequiredServices	AliasProperty	RequiredServices = ServicesDependedOn
Disposed	Event	System.EventHandler Disposed(System.Object, System.EventArgs)
Close	Method	void Close()
Continue	Method	void Continue()
CreateObjRef	Method	System.Runtime.Remoting.ObjRef CreateObjRef(type requestedType)
Dispose	Method	void Dispose(), void IDisposable.Dispose()
Equals	Method	bool Equals(System.Object obj)
ExecuteCommand	Method	void ExecuteCommand(int command)
GetHashCode	Method	int GetHashCode()
GetLifetimeService	Method	System.Object GetLifetimeService()
GetType	Method	type GetType()
InitializeLifetimeService	Method	System.Object InitializeLifetimeService()
Pause	Method	void Pause()
Refresh	Method	void Refresh()
Start	Method	void Start(), void Start(string[] args)
Stop	Method	void Stop()
WaitForStatus	Method	void WaitForStatus(System.ServiceProcess.ServiceControllerStatus desiredStat...
CanPauseAndContinue	Property	bool CanPauseAndContinue {get;}
CanShutdown	Property	bool CanShutdown {get;}
CanStop	Property	bool CanStop {get;}
Container	Property	System.ComponentModel.IContainer Container {get;}

Get-Process

- **Get-Process** Permet de récupérer la liste des processus actifs sur une machine.

```
PS C:\Users\ZITI> Get-Process
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
86	7	1040	1664		4540	0	AppleOSSMgr
344	20	13168	23188	2.06	9528	1	ApplicationFrameHost
264	14	3008	5736	2.53	10048	1	Bootcamp
257	12	4708	8556	0.50	5924	1	browserhost
568	30	28204	3792	1.13	17852	1	Calculator
562	16	45556	64604	27.86	32	1	chrome
440	16	36204	58212	4.39	1188	1	chrome
253	15	20508	37596	0.69	1464	1	chrome
322	16	37204	63512	5.17	1468	1	chrome
308	15	21628	47532	0.80	1596	1	chrome
733	19	81740	105940	124.98	1776	1	chrome
278	13	12044	18940	5.28	2216	1	chrome
244	15	21200	35232	0.48	2520	1	chrome
369	15	30372	51352	4.23	3012	1	chrome
271	16	49052	66564	19.70	3312	1	chrome
435	16	46332	74140	28.89	3564	1	chrome
507	17	48764	70972	10.72	5092	1	chrome

Get-PSDrive

La commande **Get-PSDrive** permet de récupérer des informations sur les différents lecteurs du système. Plusieurs types de lecteurs différents sont remontés par cette commande:

- Les lecteurs de type **"FileSystem"** correspondant à vos volumes montés sur votre système, par exemple le volume "C" où Windows est installé, ou alors un lecteur réseau connecté sur votre machine.
- Les lecteurs de type **"Registry"** correspondent au Registre Windows, en l'occurrence les ruches HKCU et HKLM

Name	Used (GB)	Free (GB)	Provider	Root	CurrentLocation
Alias			Alias		
C	204.39	25.53	FileSystem	C:\	Users\ZITI
Cert			Certificate	\	
D	51.81	118.97	FileSystem	D:\	
E			FileSystem	E:\	
Env			Environment		
F			FileSystem	F:\	
Function			Function		
HKCU			Registry	HKEY_CURRENT_USER	
HKLM			Registry	HKEY_LOCAL_MACHINE	
Variable			Variable		
WSMan			WSMan		

Get-Service

Grâce à la commande **Get-Service**, nous allons pouvoir récupérer des informations sur les services présents sur la machine, ainsi que sur leur état et leur type de démarrage

```
PS C:\Users\ZITI> Get-Service
```

Status	Name	DisplayName
-----	----	-----
Running	AarSvc_1bfc98	Agent Activation Runtime_1bfc98
Stopped	AJRouter	Service de routeur AllJoyn
Stopped	ALG	Service de la passerelle de la couc...
Stopped	AppIDSvc	Identité de l'application
Running	Appinfo	Informations d'application
Running	AppleOSSMgr	Apple OS Switch Manager
Stopped	AppMgmt	Gestion d'applications
Stopped	AppReadiness	Préparation des applications
Stopped	AppVClient	Microsoft App-V Client
Running	AppXSvc	Service de déploiement AppX (AppXSVC)
Stopped	AssignedAccessM...	Service AssignedAccessManager
Running	AudioEndpointBu...	Générateur de points de terminaison...
Running	Audiosrv	Audio Windows
Stopped	autotimesvc	Heure cellulaire
Stopped	AxInstSV	Programme d'installation ActiveX (A...
Stopped	BcastDVRUserSer...	GameDVR and Broadcast User Service_...
Stopped	BDESVC	Service de chiffrement de lecteur B...

Test-Connection

- L'équivalent de l'éternelle commande "ping", mais à la façon de PowerShell. Pour réaliser 4 pings à destination d'une machine, on peut spécifier son adresse IP ou son nom DNS :

```
PS C:\Users\ZITI> Test-Connection google.com
```

Source	Destination	IPv4Address	IPv6Address	Bytes	Time(ms)
-----	-----	-----	-----	----	-----
IZITI	google.com	142.251.37.174		32	38
IZITI	google.com	142.251.37.174		32	34
IZITI	google.com	142.251.37.174		32	48
IZITI	google.com	142.251.37.174		32	34

- Si l'on veut réaliser seulement 1 ping, ce qui est pratique pour simplement tester la connectivité vers une machine, on utilisera **-Count** suivi du nombre de ping à réaliser :

```
PS C:\Users\ZITI> Test-Connection google.com -Count 1
```

Source	Destination	IPv4Address	IPv6Address	Bytes	Time(ms)
-----	-----	-----	-----	----	-----
IZITI	google.com	142.251.37.174		32	41

```
PS C:\Users\ZITI>
```

Test-Connection

On peut facilement tester plusieurs hôtes : il suffit de séparer les cibles par **virgule**:

```
PS C:\Users\ZITI> Test-Connection google.com, fb.com
```

Source	Destination	IPV4Address	IPV6Address	Bytes	Time(ms)
-----	-----	-----	-----	-----	-----
IZITI	google.com	142.251.37.238		32	34
IZITI	google.com	142.251.37.238		32	34
IZITI	google.com	142.251.37.238		32	34
IZITI	google.com	142.251.37.238		32	34
IZITI	fb.com	157.240.196.35		32	37
IZITI	fb.com	157.240.196.35		32	37
IZITI	fb.com	157.240.196.35		32	34
IZITI	fb.com	157.240.196.35		32	34

-
- Gestion fichier et dossier

DOS/CMD	Équivalent PowerShell (alias)	Commandelette PowerShell	Description
DIR	DIR	Get-ChildItem	Liste le contenu d'un répertoire.
CD	CD	Set-Location	Change de répertoire courant.
MD	MD	New-Item	Crée un fichier/répertoire.
RD	RD	Remove-Item	Supprime un fichier/répertoire.
MOVE	MOVE	Move-Item	Déplace un fichier/répertoire.
REN	REN	Rename-Item	Renomme un fichier/répertoire.
COPY	COPY	Copy-Item	Copie un fichier/répertoire.

Get-Location

- Afficher le chemin du dossier courant:

```
PS C:\Users\ZITI> Get-Location
```

```
Path
```

```
----
```

```
C:\Users\ZITI
```

```
PS C:\Users\ZITI>
```

- **Alias** => "Get-Location" = "pwd"

```
PS C:\Users\ZITI> PWD
```

```
Path
```

```
----
```

```
C:\Users\ZITI
```

```
PS C:\Users\ZITI> |
```

Get-ChildItem

- Afficher le contenu du dossier courant:

```
PS C:\Users\ZITI> Get-ChildItem

Répertoire : C:\Users\ZITI
```

Mode	LastWriteTime	Length	Name
d-----	10/05/2020 22:00		.idlerc
d-----	15/02/2022 21:43		.VirtualBox
d-r----	20/04/2021 11:45		3D Objects
d-----	10/02/2021 09:28		Cisco Packet Tracer 7.3.0
d-r----	20/04/2021 11:45		Contacts
d-r----	04/07/2021 16:46		Creative Cloud Files
d-r----	06/03/2022 18:39		Desktop
d-r----	17/02/2022 11:04		Documents
d-r----	06/03/2022 18:29		Downloads
d-r----	20/04/2021 11:45		Favorites
d-r----	20/04/2021 11:45		Links
d-r----	20/04/2021 11:45		Music
d-r----	27/07/2021 12:06		OneDrive
d-r----	20/04/2021 11:45		Pictures
d-r----	20/04/2021 11:45		Saved Games
d-r----	20/04/2021 11:45		Searches
d-r----	20/04/2021 11:45		Videos
d-----	24/01/2022 20:34		VirtualBox VMs
-a-----	13/03/2021 09:06	174	.packettracer

Get-ChildItem

- Alias => "Get-ChildItem" = "ls" = "dir"

```
PS C:\Users\ZITI> ls
```

```
Répertoire : C:\Users\ZITI
```

Mode	LastWriteTime		Length	Name
d-----	10/05/2020	22:00		.idlerc
d-----	15/02/2022	21:43		.VirtualBox
d-r----	20/04/2021	11:45		3D objects
d-----	10/02/2021	09:28		Cisco Packet Tracer 7.3.0
d-r----	20/04/2021	11:45		Contacts
d-r----	04/07/2021	16:46		Creative Cloud Files
d-r----	06/03/2022	18:39		Desktop
d-r----	17/02/2022	11:04		Documents
d-r----	06/03/2022	18:29		Downloads
d-r----	20/04/2021	11:45		Favorites
d-r----	20/04/2021	11:45		Links
d-r----	20/04/2021	11:45		Music

Set-Location

- Se déplacer dans les dossiers:

```
PS C:\Users\ZITI> set-location C:\Users\ZITI\Desktop  
PS C:\Users\ZITI\Desktop> |
```

- Alias => "Set-Location" = "cd"

```
PS C:\Users\ZITI> cd C:\Users\ZITI\Desktop  
PS C:\Users\ZITI\Desktop> |
```

New-Item

- Créer un fichier vide du nom de "test.txt" dans le répertoire courant:

`New-Item -Name "test.txt" -ItemType file`

```
PS C:\Users\ZITI\Desktop> New-Item -Name "test.txt" -ItemType file
```

Répertoire : C:\Users\ZITI\Desktop

Mode	LastWriteTime	Length	Name
-a----	06/03/2022 21:29	0	test.txt

```
PS C:\Users\ZITI\Desktop> |
```

- Pour créer un fichier du nom de "test.txt" dans un autre répertoire comme par exemple dans "c:\windows", il faut rajouter le chemin dans l'option "path":

`New-Item -path "c:\windows\" -Name "test.txt" -ItemType file`

New-Item

- Créer un dossier un répertoire du nom de "dossier_de_test":

`New-Item -Name "dossier_de_test" -ItemType directory`

```
PS C:\Users\ZITI\Desktop> New-Item -Name "dossier_de_test" -ItemType directory
```

Répertoire : C:\Users\ZITI\Desktop

Mode	LastWriteTime	Length	Name
d----	06/03/2022 21:34		dossier_de_test

```
PS C:\Users\ZITI\Desktop> |
```

- Pour créer un dossier du nom de "dossier_de_test" dans un autre répertoire comme par exemple dans "c:\windows", il faut rajouter le chemin avec l'option "**path**":

`New-Item -path "c:\windows\" -Name "dossier_de_test" -ItemType directory`

Remove-Item

- Supprimer un dossier qui est dans le répertoire courant:

`Remove-Item` « dossier_de_test »

```
PS C:\Users\ZITI\Desktop> Remove-Item "dossier_de_test"
```

```
PS C:\Users\ZITI\Desktop> |
```

- Pour supprimer directement un répertoire qui contient des fichiers et des sous-dossiers sans avoir à confirmer:

`Remove-Item` "C:\Windows\dossier_de_test" `-Recurse`

- `Alias => "-Recurse" = "-r"`
- `Alias => "Remove-Item" = "rm"`

Move-Item

- Déplacer un dossier qui est dans le répertoire courant vers un autre répertoire:

Move-Item -Path "dossier_de_test" -Destination "c:\Windows\"

```
PS C:\Users\ZITI\Desktop> Move-Item -Path "dossier_de_test" -Destination "c:\windows\"  
PS C:\Users\ZITI\Desktop> |
```

- Déplacer un fichier qui est dans le répertoire courant vers un autre répertoire:

Move-Item -Path "test.txt" -Destination "c:\Windows\"

Rename-Item

- Renommer un dossier:

`Rename-Item -Path dossier_de_test -NewName dossier_de_2022`

```
PS C:\windows> Rename-Item -Path dossier_de_test -NewName dossier_de_2022
```

```
PS C:\windows> |
```

- Renommer un fichier:

`Rename-Item -Path test.txt -NewName test-2022.txt`

Copy-Item

- Copier un dossier vide:

```
Copy-Item -Path dossier_de_test -Destination c:\Windows
```

- Copier un dossier avec son contenu, il faut rajouter l'option "-Recurse":

```
Copy-Item -Path dossier_de_test -Destination c:\Windows -Recurse
```

- Copier un fichier:

```
Copy-Item -Path .\test.txt -Destination c:\Windows
```

- Si le fichier de destination existe déjà, il faut rajouter l'option "-Force":

```
Copy-Item -Path .\test.txt -Destination c:\Windows -Force
```

Test-Path

Le cmdlet **Test-Path** sert à tester l'existence d'un élément, généralement un fichier ou un dossier. Par exemple pour tester l'existence du chemin "C:\TEMP", on exécutera :

```
PS C:\Users\ZITI> Test-Path -Path "C:\TEMP"  
True  
PS C:\Users\ZITI>
```

-
- Partager des dossiers

Partager des dossiers

Nous allons voir maintenant comment créer un dossier partagé sous PowerShell. Il faut le faire comme avec l'interface graphique en 3 étapes :

1. Créez un dossier classique.
2. Créez le partage.
3. Configurez les droits de partage.

Créer le partage

- Pour partager le dossier **C:\SAUVEGARDE** en créant un nom de partage **Sauvegarde** et en donnant le contrôle total au groupe **Administrateurs**.

New-SmbShare -Name Sauvegarde -Path C:\SAUVEGARDE -FullAccess Administrators

```
\Users\ZITI\Desktop> New-SmbShare -Name Sauvegarde -Path C:\Users\ZITI\Desktop\sauvegarde -FullAccess Administrators
```

ScopeName	Path	Description
Sauvegarde	* C:\Users\ZITI\Desktop\sauvegarde	

```
\Users\ZITI\Desktop> |
```

Configurer les droits de partage

Nous pouvons modifier les droits de partage en rajoutant les droits de lecture à tout le monde en tapant la commande suivante :

`Grant-SmbShareAccess -Name Sauvegarde -AccountName "Tout le Monde" -AccessRight Read`

On peut mettre comme droit (**AccessRight**) :

- **Full** :contrôle total ;
- **Change** :modifier ;
- **Read** :lecture ;
- **Custom** :personnalisé.

Lister les partages

- Pour afficher la liste des partages :

Get-SmbShare

```
PS C:\Users\ZITI\Desktop> Get-SmbShare
```

Name	ScopeName	Path	Description
----	-----	----	-----
ADMIN\$	*	C:\WINDOWS	Remote Admin
C\$	*	C:\	Default share
D\$	*	D:\	Default share
IPC\$	*		Remote IPC
OpenSource	*	D:\OpenSource	
Sauvegarde	*	C:\Users\ZITI\Desktop\sauvegarde	
TP_Admin_Linux	*	D:\TP_Admin_Linux	
Users	*	C:\Users	
WIA_WDS_SHARE	*	C:\Program Files\Serva\install\WIA_WDS	

Lister les autorisations

- Pour lister les autorisations sur le partage:

`Get-SmbShareAccess` NomPARTAGE

```
PS C:\Users\ZITI\Desktop> Get-SmbShareAccess sauvegarde
```

Name	ScopeName	AccountName	AccessControlType	AccessRight
----	-----	-----	-----	-----
Sauvegarde	*	BUILTIN\Administrators	Allow	Full

```
PS C:\Users\ZITI\Desktop> |
```

-
- Définir une adresse IP

Commandes PowerShell	Actions
Get-NetAdapter	lister les différentes cartes réseaux physiques au sein de votre machine.
Get-NetIPAddress	lister les paramètres réseaux des cartes présents sur votre machine (Serveur ou station de travail)
Enable-NetAdapter	Activer une carte réseau
Disable-NetAdapter	Désactiver une carte réseau
New-NetIPAddress	Attribuer une adresse IP à une carte réseau
Set-DnsClientServerAddress	Configurer un DNS
Set-NetIPInterface	Modifier la configuration réseaux de la carte
Remove-NetIPAddress	Supprimer une adresse IP
Get-DnsClientServerAddress	Afficher les informations DNS de votre carte réseaux
Resolve-DnsName	Résoudre des adresses DNS
Get-NetRoute	Afficher la table route
New-NetRoute	Ajouter une nouvelle route

Liste des commandes réseaux en Powershell

Lister les cartes réseaux

Permet de repérer le **numéro d'index** de la carte que l'on souhaite modifier. Ce numéro sera utile lors de la phase de configuration.

Deux choix s'offrent à vous :

- Get-NetIPInterface (Lister les interfaces IP)
- Get-NetAdapter (Obtenir les propriétés basiques des adaptateurs réseaux)

```
PS C:\Users\ZITI> Get-NetIPInterface
```

ifIndex	InterfaceAlias	AddressFamily	NlMtu(Bytes)	InterfaceMetric	Dhcp	ConnectionState	PolicyStore
21	VirtualBox Host-Only Network	IPv6	1500	25	Enabled	Connected	A...
2	VMware Network Adapter VMnet8	IPv6	1500	35	Enabled	Connected	A...
10	VMware Network Adapter VMnet1	IPv6	1500	35	Enabled	Connected	A...
16	Bluetooth Network Connection	IPv6	1500	65	Disabled	Disconnected	A...
20	Local Area Connection* 10	IPv6	1500	25	Disabled	Disconnected	A...
14	Local Area Connection* 9	IPv6	1500	25	Disabled	Disconnected	A...
18	Ethernet	IPv6	1500	35	Enabled	Connected	A...
6	Wi-Fi	IPv6	1500	25	Enabled	Disconnected	A...
1	Loopback Pseudo-Interface 1	IPv6	4294967295	75	Disabled	Connected	A...
21	VirtualBox Host-Only Network	IPv4	1500	25	Disabled	Connected	A...
2	VMware Network Adapter VMnet8	IPv4	1500	35	Enabled	Connected	A...
10	VMware Network Adapter VMnet1	IPv4	1500	35	Enabled	Connected	A...
16	Bluetooth Network Connection	IPv4	1500	65	Enabled	Disconnected	A...
20	Local Area Connection* 10	IPv4	1500	25	Enabled	Disconnected	A...
14	Local Area Connection* 9	IPv4	1500	25	Enabled	Disconnected	A...
18	Ethernet	IPv4	1500	35	Enabled	Connected	A...
6	Wi-Fi	IPv4	1500	25	Enabled	Disconnected	A...
1	Loopback Pseudo-Interface 1	IPv4	4294967295	75	Disabled	Connected	A...

```
PS C:\Users\ZITI> Get-NetAdapter
```

	InterfaceDescription	ifIndex	Status	MacAddress	LinkSpeed
	-----	-----	-----	-----	-----
VirtualBox Host-Only N...	VirtualBox Host-Only Ethernet Adapter	21	Up	0A-00-27-00-00-15	1 Gbps
Ethernet	Broadcom NetXtreme Gigabit Ethernet	18	Up	10-DD-B1-C9-05-1B	...Mbps
Bluetooth Network Conn...	Bluetooth Device (Personal Area Netw...	16	Disconnected	28-CF-E9-65-A5-E4	3 Mbps
VMware Network Adapte...	VMware Virtual Ethernet Adapter for	10	Up	00-50-56-00-00-01	Mbps

Configuration d'une nouvelle adresse IP

Par exemple, pour ajouter l'adresse IP « 192.168.1.203/24 » à l'interface ayant l'index 12 la commande est :

```
New-NetIPAddress -InterfaceIndex 12 -IPAddress 192.168.1.203 -PrefixLength 24  
-DefaultGateway 192.168.1.254
```

- **InterfaceIndex** : Numéro d'index de la carte à modifier , il est possible d'utiliser à la place **-InterfaceAlias** suivit du nom de l'interface si vous préférez au numéro d'index)
- **IPAddress** : Adresse IP à attribuer à la carte
- **PrefixLength** : Longueur du masque de sous réseau
- **DefaultGateway** : Passerelle par défaut

Activer/Désactiver le DHCP sur une carte

Pour activer/désactiver le DHCP sur une carte utiliser la commande :

```
Set-NetIPInterface -InterfaceIndex 12 -Dhcp {Enabled/Disabled}
```

Supprimer L' adresse IP

Si une adresse IP est déjà en place sur la carte, elle ne sera pas écrasée par celle que nous venons d'ajouter, il faut donc supprimer l'ancienne.

Imaginons que sur ma carte d'index 12, j'ai une ancienne adresse IP qui est « 10.10.10.10/16 » et que je souhaite la supprimer :

```
Remove-NetIPAddress -InterfaceIndex 12 -IPAddress 10.10.10.10 -PrefixLength  
16 -DefaultGateway 10.10.10.254
```

Ajout d'un DNS

Pour ajouter un serveur DNS « 8.8.8.8 » sur notre carte

`Set-DnsClientServerAddress -InterfaceIndex 12 -ServerAddresses 8.8.8.8`

```
PS C:\WINDOWS\system32> Set-DnsClientServerAddress -InterfaceIndex 18 -ServerAddresses 8.8.8.8
PS C:\WINDOWS\system32> |
```

Pour la vérification :

`Get-DnsClientServerAddress -InterfaceIndex 12`

```
PS C:\WINDOWS\system32> Get-DnsClientServerAddress -InterfaceIndex 18

InterfaceAlias      Interface Index Address Family ServerAddresses
-----
Ethernet            18 IPv4      {8.8.8.8}
Ethernet            18 IPv6      {fe80::1, fe80::1}
```

```
PS C:\WINDOWS\system32> |
```

Vérification des paramètres

Pour vérifier que la configuration de la carte réseau est correcte, on utilise la commande suivante :

Get-NetIPConfiguration

```
PS C:\WINDOWS\system32> Get-NetIPConfiguration

InterfaceAlias      : VMware Network Adapter VMnet1
InterfaceIndex      : 10
InterfaceDescription : VMware Virtual Ethernet Adapter for VMnet1
IPv4Address          : 192.168.229.1
IPv6DefaultGateway   :
IPv4DefaultGateway   :
DNSServer            : fec0:0:0:ffff::1
                    : fec0:0:0:ffff::2
                    : fec0:0:0:ffff::3

InterfaceAlias      : Ethernet
InterfaceIndex      : 18
InterfaceDescription : Broadcom NetXtreme Gigabit Ethernet
NetProfile.Name      : ZTE-8756c8
IPv4Address          : 190.168.10.31
IPv6DefaultGateway   : fe80::1
IPv4DefaultGateway   : 190.168.10.1
DNSServer            : fe80::1
                    : fe80::1
                    : 8.8.8.8

InterfaceAlias      : VMware Network Adapter VMnet8
InterfaceIndex      : 2
InterfaceDescription : VMware Virtual Ethernet Adapter for VMnet8
IPv4Address          : 192.168.230.1
IPv6DefaultGateway   :
IPv4DefaultGateway   :
DNSServer            : fec0:0:0:ffff::1
```

-
- Renommer une machine Windows

Renommer une machine Windows

Pour renommer le serveur avec **Rename-Computer**, il suffit de spécifier le paramètre **-NewName**, suivi du nom que l'on souhaite, cela donne la commande suivante :

Rename-Computer -NewName « PC01 »

Pour la prise en charge de la modification il faut redémarrer le serveur :

Restart-Computer

Au redémarrage de la machine, on peut vérifier que le nom est changé grâce à la commande **hostname** .

```
PS C:\Users\ZITI> hostname  
IZITI  
  
PS C:\Users\ZITI> |
```

Get-WmiObject

- Parametre machine
- Cpu
- Ram D D

Bios

Pour récupérer les information du BIOS utiliser la commande :

Get-WmiObject -Class Win32_BIOS

```
PS C:\WINDOWS\system32> Get-WmiObject -Class Win32_BIOS
```

```
SMBIOSBIOSVersion : MBP91.88Z.00D7.B00.1708080744  
Manufacturer      : Apple Inc.  
Name              : Default System BIOS  
SerialNumber      : C02KLOE6DV33  
Version           : APPLE - 0
```

```
PS C:\WINDOWS\system32> |
```

Processeur

Pour récupérer les information du Processeur utiliser la commande:

Get-WmiObject -Class Win32_Processor

```
PS C:\WINDOWS\system32> Get-WmiObject -Class Win32_Processor
```

```
Caption           : Intel64 Family 6 Model 58 Stepping 9
DeviceID          : CPU0
Manufacturer      : GenuineIntel
MaxClockSpeed     : 2301
Name              : Intel(R) Core(TM) i7-3615QM CPU @ 2.30GHz
SocketDesignation : U2E1
```

Système

Pour récupérer les information du Système utiliser la commande:

Get-WmiObject -Class Win32_ComputerSystem

```
PS C:\WINDOWS\system32> Get-WmiObject -Class Win32_ComputerSystem

Domain                : WORKGROUP
Manufacturer          : Apple Inc.
Model                 : MacBookPro9,1
Name                  : IZITI
PrimaryOwnerName      : ZITI
TotalPhysicalMemory   : 8525639680
```

Systeme d'exploitation

Pour récupérer les information du Systeme d'exploitation utiliser la commande:

Get-WmiObject -Class Win32_OperatingSystem

```
PS C:\WINDOWS\system32> Get-WmiObject -Class win32_operatingSystem
```

```
SystemDirectory : C:\WINDOWS\system32
Organization    :
BuildNumber     : 19044
RegisteredUser  : ZITI
SerialNumber    : 00331-10000-00001-AA244
Version        : 10.0.19044
```

Espace disque

Pour récupérer les information du Espace disque utiliser la commande:

Get-WmiObject -Class Win32_LogicalDisk

```
PS C:\WINDOWS\system32> Get-WmiObject -Class Win32_LogicalDisk
```

```
DeviceID      : C:  
DriveType     : 3  
ProviderName  :  
FreeSpace     : 52225822720  
Size          : 246878892032  
VolumeName    : BOOTCAMP
```

```
DeviceID      : D:  
DriveType     : 3  
ProviderName  :  
FreeSpace     : 122842185728  
Size          : 183371821056  
VolumeName    : New Volume
```

```
DeviceID      : E:  
DriveType     : 5  
ProviderName  :  
FreeSpace     :  
Size          :  
VolumeName    :
```

-
- Manipuler la date et l'heure

Récupérer la date

Pour récupérer la date la commande est :

Get-Date

```
PS C:\Users\amina> Get-date  
samedi 30 mars 2024 14:50:51
```

Remarque : Avec le paramètre *Format*, nous allons pouvoir le personnaliser en jouant avec les lettres "m", "d", "y", "h", "s", "f" où la casse est importante et impacte le résultat retourné.

```
PS C:\Users\amina> Get-Date -Format "dd/MM/yyyy"  
30/03/2024  
PS C:\Users\amina>
```

Calculer une date avec AddDays

Pour calculer une date, nous allons utiliser la méthode `AddDays()` qui s'applique directement à `Get-Date` et qui attend un nombre comme valeur. Si le nombre est positif, le nombre de jours sera ajouté à la date du jour, s'il est négatif, il sera soustrait et la date calculée sera retournée.

Pour soustraire 90 jours à la date du jour, cela nous donne :

`(Get-Date).AddDays(-90)`

```
PS C:\Users\amina> Get-date
```

```
samedi 30 mars 2024 14:53:37
```

```
PS C:\Users\amina> (Get-Date).AddDays(-90)
```

```
dimanche 31 décembre 2023 14:53:46
```

Calculer une date avec AddDays

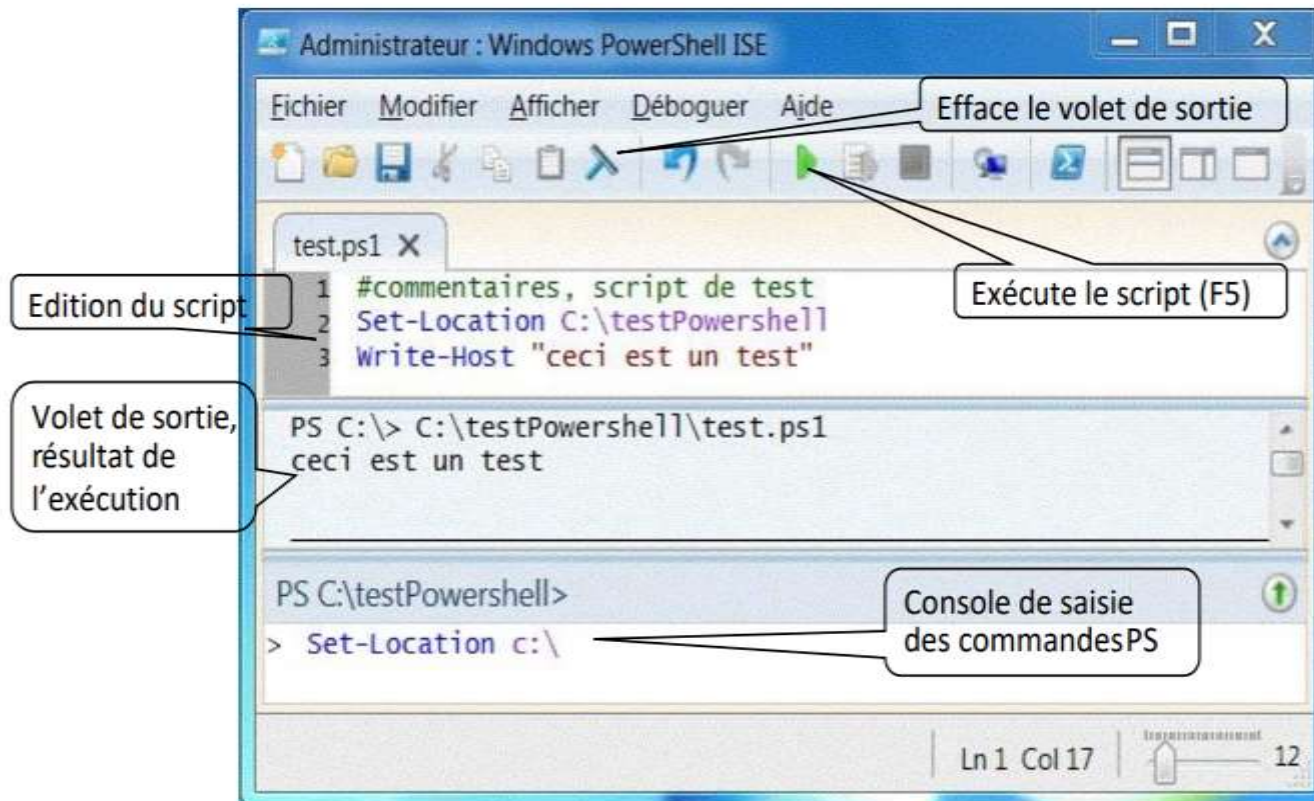
Exemple:

```
Write-Output "Date du jour : $(Get-Date)"  
Write-Output "Date il y a 90 jours : $((Get-Date).AddDays(-90))"  
Write-Output "Date dans 90 jours : $((Get-Date).AddDays(90))"
```

```
PS C:\Windows\system32> C:\Users\amina\Downloads\cours eniad\eniad final\chap 2\exemple1.ps1  
Date du jour : 03/30/2024 15:14:38  
Date il y a 90 jours : 12/31/2023 15:14:38  
Date dans 90 jours : 06/28/2024 15:14:38
```

Calculer une date avec AddDays

Exemple:



Calculer une date avec AddDays

Dans PowerShell, il existe quatre paramètres de stratégie d'exécution des scripts qui sont :

- **Restricted** : paramètre par défaut, n'autorise pas l'exécution des scripts,
- **AllSigned** : n'exécute que les scripts de confiance, donc signés,
- **RemoteSigned** : exécute les scripts locaux sans obligation de confiance et les scripts de confiance issus d'Internet,
- **Unrestricted** : autorise l'exécution de tous les scripts

```
>Set-ExecutionPolicy RemoteSigned
```

Modifier la date et l'heure

- La commande **Set-Date** permet de modifier la date et l'heure sur notre système.
- Pour utiliser cette commande. On va utiliser le paramètre **-Date** et préciser la date et l'heure souhaitée. Exemple:

Set-Date -Date " 15/05/2022 17:52"

```
PS C:\WINDOWS\system32> Set-Date -Date " 15/05/2022 17:52"
```

```
dimanche 15 mai 2022 17:52:00
```

```
PS C:\WINDOWS\system32> Get-Date
```

```
dimanche 15 mai 2022 17:52:08
```

```
PS C:\WINDOWS\system32> |
```

Modifier la date et l'heure

- Pour changer seulement la date

Set-Date -Date "27/09/2022"

- De la même façon, on peut ajuster l'heure sans modifier la date :

Set-Date -Date "18:00"

-
- Gestion des paquets

Introduction

- PowerShell Package Management vous permet de rechercher, d'installer, de mettre à jour et de désinstaller des modules PowerShell et d'autres packages.
- PowerShellGallery.com est la source par défaut pour les modules PowerShell. Vous pouvez également parcourir le site pour rechercher les packages disponibles, commander et prévisualiser le code.

Trouver un module par nom

Find-Module -Name <Name>

```
PS C:\Users\ntic> Find-Module -Name PowerVMDS
```

Version	Name	Repository	Description
1.0	PowerVMDS	PSGallery	Powershell Module for the VMDSC Fling API

```
PS C:\Users\ntic> |
```

Trouver un module par nom

Pour trouver un module qui se termine par DSC par exemple

```
Find-Module -Name *DSC
```

```
PS C:\Users\ntic> Find-Module -Name *DSC
```

Version	Name	Repository	Description
8.2.0	NetworkingDsc	PSGallery	DSC resources for configuring settings related to networking.
8.5.0	ComputerManagementDsc	PSGallery	DSC resources for configuration of a windows computer. These DSC resources allow you to perform comput...
3.1.1	WSManDsc	PSGallery	DSC resources for configuring WS-Man.
5.1.0	CertificateDsc	PSGallery	DSC resources for managing certificates on a windows Server.
15.2.0	SqlServerDsc	PSGallery	Module with DSC resources for deployment and configuration of Microsoft SQL Server.
1.2.0.0	SystemLocaleDsc	PSGallery	This DSC Resource allows configuration of the windows System Locale.
2.0.0.9	ReverseDsc	PSGallery	This DSC module is used to extract the DSC Configuration of existing environments.
1.21.11...	Microsoft365Dsc	PSGallery	This DSC module is used to configure and monitor Microsoft tenants, including SharePoint Online, Excha...
5.0.1	StorageDsc	PSGallery	DSC resources for managing storage on windows Servers.
4.9.0	SharePointDsc	PSGallery	This DSC module is used to deploy and configure SharePoint Server 2013, 2016 and 2019, and covers a wi...
6.0.1	ActiveDirectoryDsc	PSGallery	The ActiveDirectoryDsc module contains DSC resources for deployment and configuration of Active Direct...
1.4.0.0	AuditPolicyDsc	PSGallery	The AuditPolicyDsc module allows you to configure and manage the advanced audit policy on all currentl...
2.10.0.0	SecurityPolicyDsc	PSGallery	This module is a wrapper around secedit.exe which provides the ability to configure user rights assign...
1.5.0	OfficeOnlineServerDsc	PSGallery	The officeonlineserverdsc module provides the ability to install office online Server (formerly known ...
5.0.0	ActiveDirectoryCSDsc	PSGallery	DSC resources for installing, uninstalling and configuring Certificate Services components in windows ...
1.2.0	GPRegistryPolicyDsc	PSGallery	This resource module contains DSC resources used to apply and manage local group policies by modifying...
1.2.1	UpdateServicesDsc	PSGallery	Module with DSC Resources for deployment and configuration of windows Server Update Services.
1.4.2	AccessControlDsc	PSGallery	Module with DSC resource to manage Registry and NTFS access entries and manage Active Directory SACL

Installer/ Désinstaller un module

- Installer un module par nom

```
Install-Module -Name <name>
```

```
PS C:\WINDOWS\system32> Install-Module -Name PowerVMDS  
PS C:\WINDOWS\system32> |
```

- Désinstaller un module

```
Uninstall-Module -Name <Name>
```

```
PS C:\WINDOWS\system32> unInstall-Module -Name PowerVMDS  
PS C:\WINDOWS\system32> |
```

Mettre à jour un module

- Pour mettre à jour un module par nom utiliser la commande:

```
Update-Module -Name <Name>
```

-
- Les Conditions

-
- Un bloc de structure conditionnelle commence toujours par une instruction **"If"** et elle sera éventuellement suivie par une ou plusieurs instructions **Else/Elseif**.
 - On peut traduire ces trois instructions :
 - If = Si...
 - Else = Sinon...
 - Elseif = Sinon si...

On peut avoir une instruction "If" seule mais ce n'est pas possible pour les instructions Else et Elseif.

If et Elseif peuvent inclure une ou plusieurs conditions, alors que Else n'accepte pas de condition (il s'applique lorsque toutes les conditions sont fausses).

Condition If

- La syntaxe d'une structure conditionnelle If simple est la suivante :

```
if (condition)
{
# bloc de code (instructions)
}
```

- La partie "condition" va permettre de tester une ou plusieurs conditions. Dans le cas où la condition se vérifie, alors le bloc de code sera exécuté.

Ce bloc de code peut contenir un autre If (conditions If imbriquées), une boucle d'un autre type, ou tout autre commande : il n'y a pas de restriction à ce niveau-là.

Condition If

Exemple : Le script suivant permet de déterminer si une valeur entrée par l'utilisateur est la lettre A

```
$var = Read-Host "Entrez un caractère"  
If ($var -eq 'A')  
{  
Write-Output "Le caractère saisi par l'utilisateur est un 'A'"  
}
```

- Le résultat si on entre la valeur A est

```
PS C:\Users\ntic> C:\Users\ntic\Desktop\PI.ps1  
Entrez un caractère : A  
Le caractère saisi par l'utilisateur est un 'A'  
  
PS C:\Users\ntic> |
```

- Si on entre la valeur B est

```
PS C:\Users\ntic> C:\Users\ntic\Desktop\PI.ps1  
Entrez un caractère : B  
  
PS C:\Users\ntic> |
```

Condition If / Else

- À l'instruction **If** peut être associée la clause **Else**. Cette clause permet en cas de retour d'une valeur **False** d'orienter le traitement vers un second bloc d'instructions.

```
If (condition)
{
    # Bloc d'instructions 1
}
Else
{
    # Bloc d'instructions 2
}
```

Condition If / Else

Prenons l'exemple de tout à l'heure, on va ajouter une condition supplémentaire à cette phrase : *Si \$var est également à A*

```
$var = Read-Host "Entrez un caractère"
If ($var -eq 'A')
{
Write-Output "Le caractère saisi par l'utilisateur est un 'A'"
}
else {
Write-Output "Oups !"
}
```

```
PS C:\Users\ntic> C:\Users\ntic\Desktop\PI.ps1
Entrez un caractère : A
Le caractère saisi par l'utilisateur est un 'A'

PS C:\Users\ntic> C:\Users\ntic\Desktop\PI.ps1
Entrez un caractère : B
Oups !

PS C:\Users\ntic>
```

Condition If / Elself / Else

Elself est destiné à être utilisé avec **If**. Il indique que « l'expression dans cette construction sera évaluée si l'instruction If précédente a pour valeur False ». Comme **If**, **Elself** peut accepter une expression. Il est donc pratique de considérer **Elself** comme un **if secondaire**.

```
If (condition)
{
    # Bloc d'instructions 1
}
Elself (condition)
{
    # Bloc d'instructions 2
}
Else
{
    # Bloc d'instructions 3
}
```

Condition If / Elseif / Else

```
$var = Read-Host "Entrez un caractère"
If ($var -eq 'A')
{
Write-Output "Le caractère saisi par l'utilisateur est un 'A'"
}
Elseif ($var -eq 'B')
{
Write-Output "Le caractère saisi par l'utilisateur est un 'B'"
}
else {
Write-Output "Oups !"
}
```

```
PS C:\Users\ntic> C:\Users\ntic\Desktop\PI.ps1
Entrez un caractère : A
Le caractère saisi par l'utilisateur est un 'A'

PS C:\Users\ntic> C:\Users\ntic\Desktop\PI.ps1
Entrez un caractère : B
Le caractère saisi par l'utilisateur est un 'B'

PS C:\Users\ntic> C:\Users\ntic\Desktop\PI.ps1
Entrez un caractère : C
Oups !
75
PS C:\Users\ntic> |
```

Double condition avec If et -or / -and

- L'exemple précédent à base If/Else/Else a l'avantage de permettre d'exécuter un bloc de code différent en fonction de si \$var= 'A' ou \$var = 'B'. Nous pourrions avoir besoin d'exécuter le même code que \$var soit **égal** à 'A' **ou** 'B'.
- Il suffit d'inclure deux conditions à tester dans l'instruction **"If" à l'aide -or ("ou")**. Ceci va permettre de tester si l'une des deux conditions est vraie.
- Si vous souhaitez que les **deux soient vraies**, vous devrez utiliser **-and (et)** : tout en sachant que le mix des deux est possible.

```
$var = Read-Host "Entrez un caractère"
If (($var -eq 'A') -or ($var -eq 'B'))
{
Write-Output "Le caractère saisi par l'utilisateur est un 'A' ou 'B'"
}
else {
Write-Output "Oups !"
}
```

Double condition avec If et -or / -and

- Voici un autre exemple avec "-and" où l'on va tester la valeur de deux variables.

```
$compteur1 = Read-Host "Entrez le premier numéro"  
$compteur2 = Read-Host "Entrez deuxième numéro"  
If (($compteur1 -eq 10) -and ($compteur2 -eq 20))  
{  
Write-Output "Les deux compteurs sont valides !"  
}
```

```
PS C:\Users\ntic> C:\Users\ntic\Desktop\PI.ps1  
Entrez le premier numéro : 10  
Entrez deuxième numéro : 20  
Les deux compteurs sont valides !  
  
PS C:\Users\ntic> |
```

opérateurs de comparaison

Opérateur	Définition
-eq	est égal à
-ne	n'est pas égal à
-gt	est supérieur à
-ge	est supérieur ou égal à
-lt	est inférieur à
-le	est inférieur ou égal à

- **Les Boucles**

Introduction

- Une boucle est une séquence d'instructions répétée continuellement jusqu'à ce qu'une certaine condition soit atteinte.
- existe exactement 5 boucles différentes en PowerShell, 4 d'entre elles sont issue de .NET et une est une cmdlet, j'ai nommé: **foreach**, **while**, **do while**, **for** et **ForEach-Object**

FOR (Pour)

- En PowerShell, la boucle "For" s'appuie sur la syntaxe suivante :

```
For (<état initial>;<condition de répétition>;<incrément>)  
{  
<Si la condition est vraie, on exécute ce bloc d'instructions>  
}  
<Si la condition est fausse, la boucle for se termine et le script continue...>
```

- Lorsque l'on utilise une boucle for, on suit la logique suivante : on indique une valeur **de départ (état initial)**, une valeur **cible** dans la condition de répétition et on **incrémente** la valeur à chaque tour de boucle (à chaque itération) : on peut incrémenter de 1, de 2, de 10, etc... au choix.

FOR (Pour)

Exemple: On peut se dire que l'on part de la valeur 0 et que, tant que la valeur est inférieure strictement à 5, on exécute le bloc d'instruction de la boucle "For", en augmentant de +1 à chaque itération.

```
for ($i = 0; $i -lt 5; $i ++)  
{  
Write-Output "La valeur est $i "  
}
```

Ce qui va donner le résultat suivant :

```
PS C:\Users\ntic> C:\Users\ntic\Desktop\PI.ps1  
La valeur est 0  
La valeur est 1  
La valeur est 2  
La valeur est 3  
La valeur est 4  
  
PS C:\Users\ntic> |
```

FOR (Pour)

- **Exemple :** Imaginons que l'on ait un tableau de chaînes de caractères (*string*) et que l'on souhaite exécuter une action sur chaque valeur du tableau, sans connaître à l'avance le nombre d'éléments du tableau, on peut s'appuyer sur la propriété *Length*.
- On va définir la variable `$langages` et lui attribuer des valeurs (cela pourrait être dynamique). Grâce à `$langages.Length`

```
$langages = @("PowerShell","Python","PHP","JavaScript","C-Sharp")  
For ($i=0;$i -lt $langages.Length;$i++)  
{  
Write-Output "Le meilleur langage de programmation est $($langages[$i])"  
}
```

```
PS C:\Users\ntic> C:\Users\ntic\Desktop\PI.ps1  
Le meilleur langage de programmation est PowerShell  
Le meilleur langage de programmation est Python  
Le meilleur langage de programmation est PHP  
Le meilleur langage de programmation est Javascript  
Le meilleur langage de programmation est C-Sharp
```

```
PS C:\Users\ntic> |
```

ForEach

ForEach a deux significations différentes dans PowerShell. L'un est un **mot-clé** et l'autre est un alias pour l' **applet de commande ForEach-Object**. Le premier est décrit ici.

Cet exemple montre comment imprimer tous les éléments d'un tableau sur l'hôte de la console:

```
$Names = @('Amy', 'Bob', 'Celine', 'David')
ForEach ($Nom in $Names)
{
    Write-Host "Hi, my name is $Nom!"
}
```

```
PS C:\Users\ntic> C:\Users\ntic\Desktop\PI.ps1
Hi, my name is Amy!
Hi, my name is Bob!
Hi, my name is Celine!
Hi, my name is David!

PS C:\Users\ntic> |
```

while (Tandis que)

- Une boucle **while** évalue une condition et si true exécute une action. Tant que la condition est vraie, l'action continuera d'être effectuée.

```
while (condition)
{
code_block
}
```

while (Tandis que)

L'exemple suivant crée une boucle qui comptera de 10 à 0

```
$i = 10  
while ($i -ge 0)  
{  
    $i  
    $i--  
}
```

```
PS C:\Users\ntic> C:\Users\ntic\Desktop\PI.ps1  
10  
9  
8  
7  
6  
5  
4  
3  
2  
1  
0  
  
PS C:\Users\ntic> |
```

Do-Loops (Faire)

Les **Do-Loops** sont utiles lorsque vous voulez toujours exécuter au moins une fois un bloc de code. Une **Do-Loops** évaluera la condition après l'exécution du codeblock, contrairement à une boucle while qui le fait avant d'exécuter le codeblock. Vous pouvez utiliser les boucles de deux manières:

- Boucle tant *que* la condition est vraie:

```
Do
{
  code_block
} while (condition)
```

- Boucle *jusqu'à ce que* la condition soit vraie, en d'autres termes, boucle pendant que la condition est fausse:

```
Do
{
  code_block
} until (condition)
```

Do-Loops (Faire)

- Exemple:

```
$i = 0  
Do {  
    $i++  
    "Number $i"  
} while ($i -ne 3)
```

```
PS C:\Users\ntic> C:\Users\ntic\Desktop\PI.ps1  
Number 1  
Number 2  
Number 3  
  
PS C:\Users\ntic>
```

ForEach

- Cet exemple montre comment capturer la sortie d'une boucle ForEach:

```
$Numbers = ForEach ($Nombre in 1..20)
{
$Nombre # Alternatively, Write-Output $Number
}
```

```
$Numbers = @() ForEach ($Number in 1..20)
{
$Numbers += $Number
}
```

ForEach-Object

- L' ForEach-Object fonctionne de manière similaire à l'instruction foreach , mais tire son entrée du pipeline.

```
$object | ForEach-Object  
{  
    code_block  
}
```

```
$names = @("Any","Bob","Celine","David")  
$names | ForEach-Object  
{  
    "Hi, my name is $_!"  
}
```

Continuer

L'opérateur Continue fonctionne dans les boucles **For** , **ForEach** , **While** et **Do** Il ignore l'itération actuelle de la boucle, en sautant au sommet de la boucle la plus interne.

```
$i =0
while ($i -lt 20) {
    $i++
    if ($i -eq 7) { continue }
    Write-Host $i
}
```

Le résultat ci-dessus produira 1 à 20 sur la console mais manquera le numéro 7.

```
PS C:\Users\ntic> C:\Users\ntic\Desktop\PI.ps1
1
2
3
4
5
6
8
9
10
11
12
13
14
15
16
17
18
19
20
```

Pause

L'opérateur de break sortira immédiatement d'une boucle de programme. Il peut être utilisé dans les boucles For ,ForEach ,While et Do ou dans une instruction Switch .

```
$i = 0
while ($i -lt 15) {
    $i++
    if ($i -eq 7) {break}
    Write-Host $i
}
```

```
PS C:\Users\ntic> C:\Users\ntic\Desktop\PI.ps1
1
2
3
4
5
6

PS C:\Users\ntic>
```